Université Cadi Ayyad
École Nationale des Sciences Appliquées de Safi
Dépt. Génie Informatique, Réseaux et Télécommunications

Ahmed Bentajer

a.bentajer@uca.ac.ma

Manuel TD/TP
Modules : Programmation Java et Modélisation UML

Table des matières

I Programmation Java Procédural **3**

II Programmation Java Orienté Objet (OO) **7**

III Modélisation UML **18**

Première partie
Programmation Java Procédural

TP 1 : Instructions de base (JAVA – Hello World)

Exercice 1

Écrire un programme qui saisit 2 entiers et affiche successivement la somme, la différence, le produit et le quotient de ces 2 entiers.

Exercice 2

Écrire un programme qui demande à l'utilisateur de donner le rayon d'un cercle et lui affiche sa surface et son périmètre.

Exercice 3

Écrire un programme qui saisit deux entiers a et b et permute la valeur de ces deux entiers.

Exercice 4

Écrire un programme qui saisit un réel x et un entier n et affiche x à la puissance n.

Exercice 5

Écrire un programme retournant la factorielle d'un nombre entier entré par l'utilisateur.

Exercice 6

Écrire un programme qui résout les équations du premier degré ($ax + b = 0$). Le programme demande a et b à l'utilisateur puis indique le nombre de solutions ainsi que leurs valeurs.

Exercice 7

Écrire un programme qui résout les équations du second degré ($ax^2 + bx + c = 0$). Le programme demande a, b et c à l'utilisateur puis indique le nombre de solutions ainsi que leurs valeurs.

Exercice 8

Écrire un programme retournant le PGCD de 2 entiers entrés par l'utilisateur.

TP 2 : Structures Itératives (JAVA – Boucles)

Exercice 1

Écrivez un programme qui lit N nombres entiers au clavier et qui affiche leur somme, leur produit et leur moyenne. Choisissez un type approprié pour les valeurs à afficher. Le nombre N est à entrer au clavier. Résolvez ce problème :

— En utilisant `while`,
— En utilisant `do - while`,
— En utilisant `for`.

Exercice 2

Calculez par multiplications successives X^N de deux entiers naturels X et N entrés au clavier.

Exercice 3

Calculez la somme des N premiers termes de la série harmonique : $1 + 1/2 + 1/3 + ... + 1/N$

Exercice 4

Affichez un triangle formé d'étoiles de N lignes (N est fourni au clavier) :
Nombre de lignes : 5

```
*
**
***
****
*****
```

Exercice 5

Écrire un programme qui calcule n ! (factoriel) une fois avec un `while` puis une seconde fois avec un `for`.

Exercice 6

Écrire un programme qui teste si un nombre est premier ou pas, puis un programme qui teste tous les nombres entre 1 et N.

TP 3 : Les Tableaux (JAVA – Tableaux)

Exercice 1

Écrire un programme dont lequel vous déclarer et initialisez un tableau puis vous affichez les éléments du tableau.

Exercice 2

Écrire un programme qui demande à l'utilisateur de déclarer un tableau de 10 éléments, puis remplir le tableau et l'afficher.

Exercice 3

Écrire un programme qui lit la taille N d'un tableau T du type `int` (dimension maximale : 50, vérifier la taille et demandez à l'utilisateur de saisir la taille tant que la valeur n'est pas comprise entre 10 et 50), remplit le tableau par des valeurs entrées au clavier et l'afficher.

Exercice 4

Écrire un programme qui lit la taille N d'un tableau T du type `int` (dimension maximale : 50, vérifier la taille et demandez à l'utilisateur de saisir la taille tant que la valeur n'est pas comprise entre 10 et 50), remplit le tableau par des valeurs entrées au clavier et l'afficher. Effacez ensuite les occurrences de 5 et tassez les éléments restants. Afficher le tableau résultant.

Exercice 5

Écrire un programme qui lit la taille N d'un tableau T du type `int`, remplit le tableau par des valeurs entrées au clavier et affiche le tableau. Ranger ensuite les éléments du tableau T dans l'ordre inverse sans utiliser de tableau d'aide. Afficher le tableau résultant.

Exercice 6

Écrire un programme qui lit la taille N d'un tableau T du type `int`, remplit le tableau par des valeurs entrées au clavier et affiche le tableau. Copiez ensuite toutes les composantes strictement positives dans un deuxième tableau TP et toutes les valeurs strictement négatives dans un troisième tableau TN. Afficher les tableaux TP et TN.

Exercice 7

Écrire un programme qui :
- Demande à l'utilisateur de saisir la taille du tableau.
- Crée un objet tableau avec la taille entrée par l'utilisateur.
- Demande à l'utilisateur de remplir le tableau.
- Calcule la somme des éléments saisis.
- Calcule la moyenne des éléments.
- Copier le tableau dans un deuxième tableau T2 et l'afficher.
- Classer les éléments du tableau dans un ordre croissant.

Exemple : taille = 5
Somme = 5 + 10 + 2 + 17 + 23 = 57
Tableau initial :

5	10	2	17	23

Tableau trié :

2	5	10	17	23

Exercice 24

Écrire un programme qui calcule le plus grand écart dans un tableau (l'écart est la valeur absolue de la différence de deux éléments).

Deuxième partie

Programmation Java Orienté Objet (OO)

TP 1 : Classes et Objets

Exercice 1

Soit la classe Personne définit par les champs suivant :
— CIN : String
— Nom : String
— Prénom : String
— Email : String
— Age : Int

1. Écrire le code pour créer la class Personne
2. Écrire la fonction initialiser qui permet d'initialiser les valeur d'un objet personne
3. Écrire une fonction afficherPersonne qui permet d'afficher les informations d'une personne
4. Écrire une fonction main pour implémenter un exemple d'utilisation

Exercice 2

Réaliser une classe Point permettant de représenter un point sur un axe. Chaque Point sera caractérisé par un nom (de type char) et une abscisse (de type double). On prévoira :
— Une fonction initialiser recevant en arguments le nom et l'abscisse d'un point,
— Une deuxième fonction initialiser recevant en argument un objet de type Point
— une méthode affiche imprimant le nom du point et son abscisse,
— une méthode translate effectuant une translation définie par la valeur de son argument.
Écrire un petit programme utilisant cette classe pour créer un point, en afficher les caractéristiques, le déplacer et en afficher à nouveau les caractéristiques.

Exercice 3

Écrire une classe Livre avec les attributs suivants : titre, auteur, nombre de pages et prix. Cette classe contiendra deux constructeurs :
— Un constructeur qui prend en paramètre le titre, l'auteur et le nombre de pages.
— Un deuxième constructeur qui a un paramètre de plus qui est le prix.
Des méthodes d'accès et d'altération (get et set) pour les différents attributs. Une méthode toString() qui renvoie une chaîne de caractères qui décrit le livre. Si le prix d'un livre n'a pas été donné, la description du livre (toString()) devra indiquer «Prix pas encore donné ». Un livre peut être gratuit (0 est une valeur possible pour un prix) ; la valeur -1 indiquera que le prix n'a pas encore été donné.

On bloque complètement les prix : un prix ne peut être donné qu'une seule fois et ne peut être modifié ensuite (une tentative pour changer le prix ne fait qu'afficher un

message d'erreur). Vous ajouterez une variable booléenne prixFixe qui indiquera que le prix ne peut plus être modifié. Écrivez une méthode « prixFixe » qui renvoie si le prix a déjà été fixé. Écrivez une méthode « compare » pour comparer 2 livres sur leur nombre de pages. Cette méthode prend un livre en paramètre, et elle renvoie 0 si les deux livres ont le même nombre de pages, 1 si l'instance courante (this) a plus de pages que le paramètre et -1 sinon.

Créez une classe Etagere pour représenter une étagère qui peut contenir un certain nombre de livres (fixe pour chaque étagère). Vous utiliserez pour cela un tableau. Le constructeur prendra en paramètre le nombre de livres que pourra contenir l'étagère. Vous ajouterez des méthodes pour :

- Donner le nombre de livres que peut contenir l'étagère, et le nombre de livres qu'elle contient.
- Ajouter des livres (« void ajouter (Livre) »). Vous ajouterez les livres « à la fin » de l'étagère. Il devra être impossible d'ajouter des livres dans une étagère pleine.
- Récupérer un livre dont on donne la position sur l'étagère (le livre reste sur l'étagère, on récupère simplement les informations sur le livre). La méthode renverra une instance de livre. La position du premier livre d'une étagère devra être 1 (et pas 0, bien que le livre soit rangé dans la première position du tableau, qui est d'indice 0). La signature de la méthode sera « Livre getLivre(int) ».
- Chercher sur l'étagère un livre repéré par son titre et son auteur. La méthode renverra la position du livre dans l'étagère (ou 0 si le livre n'y est pas).
- Supprimer des livres.

TP 2 : Encapsulation et Constructeurs

Exercice 1

Définir une classe Rectangle ayant les attributs suivants : Longueur et Largeur.

- Définir à l'aide des propriétés les méthodes d'accès aux attributs de la classe.
- Ajouter un constructeur d'initialisation.
- Ajouter les méthodes suivantes :
  - Périmètre () : retourne le périmètre du rectangle.
  - Aire() : retourne l'aire du rectangle.
  - EstCarre() : vérifie si le rectangle est un carré.
  - AfficherRectangle() : expose les caractéristiques d'un rectangle comme suit : Longueur : [...] - Largeur : [...] - Périmètre : [...] - Aire : [...] - Il s'agit d'un carré / Il ne s'agit pas d'un carré

Exercice 2

Écrire une classe Complexes permettant de représenter des nombres complexes. Un nombre complexe Z comporte une partie réelle et une partie imaginaire : $Z = PartieRelle + PartieImaginaire \times i$

- Définir à l'aide des propriétés les méthodes d'accès aux attributs de la classe.
- Définir un constructeur par défaut permettant d'initialiser les deux parties du nombre à 0.
- Définir un constructeur d'initialisation pour la classe.
- Ajouter les méthodes suivantes :
  - Plus(Complexe) : Elle permet de retourner le nombre complexe obtenu en ajoutant au nombre en cours un nombre complexe passé en argument.
  - Moins(Complexe) : Elle permet de retourner le nombre complexe obtenu en soustrayant au nombre en cours un nombre complexe passé en argument.
  - Afficher () : Elle donne une représentation d'un nombre complexe comme suit : a+b*i.
- Écrire un programme permettant de tester la classe Complexe.

Exercice 3

Définir une classe Employé caractérisée par les attributs : Matricule , Nom , Prénom , DateNaissance , DateEmbauche , Salaire . (Date naissance et Date embauche sont des objet de type DateFormat ayant les attribut jour, mois et année)

- Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe.
- Définir un constructeur permettant d'initialiser les attributs de la méthode par des valeurs saisies par l'utilisateur.
- Ajouter à la classe la méthode Age() qui retourne l'âge de l'employé.
- Ajouter à la classe la méthode Anciennete() qui retourne le nombre d'années d'ancienneté de l'employé.
- Ajouter à la classe la méthode AugmentationDuSalaire() qui augmente le salaire de l'employé en prenant en considération l'ancienneté. Si Ancienneté < 5 ans, alors on ajoute 2%. Si Ancienneté < 10 ans, alors on ajoute 5%. Sinon, on ajoute 10%.

- Ajouter la méthode AfficherEmployé() qui affiche les informations de l'employé comme suit : Matricule : [...] - Nom complet : [NOM Prénom] - Age : [...] - Ancienneté : [...] - Salaire : [...]
- Écrire un programme de test pour la classe Employé.

Exercice 4

Créer la classe Article caractérisée par les attributs : Référence, Désignation, PrixHT, TauxTVA. Ces attributs doivent seulement être accessibles par le biais des accesseurs (get / set) en lecture/écriture mis en œuvre par les propriétés.

- Ajouter les constructeurs suivants : Un constructeur par défaut, Un constructeur initialisant tous les attributs, Un Constructeur qui permet de renseigner la référence et la désignation lors de l'instanciation, Un constructeur de recopie.
- Implémentez la méthode CalculerPrixTTC() ; Cette méthode doit calculer le prix TTC d'un article qui équivaut à : $PrixHT + (PrixHT \times \frac{TauxTVA}{100})$ et retournera la valeur calculée.
- Ajouter la méthode AfficherArticle() qui affiche les informations de l'article.
- Créer un programme de test où il faut créer des objets (en utilisant les différents constructeurs) et leur calculer le prix TTC.
- Le taux de TVA est en fait commun à tous les articles. Pour éviter toute redondance de cet attribut, vous devriez donc la déclarer comme partagée au niveau de la classe Article et non comme un attribut spécifique des objets instanciés à partir de la classe. Proposer une solution et tester de nouveau.

Exercice 5

Définir une classe Client avec les attributs suivants : CIN, Nom, Prénom, Tél.

- Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe.
- Définir un constructeur permettant d'initialiser tous les attributs.
- Définir un constructeur
- Définir la méthode Afficher () permettant d'afficher les informations du Client en cours.

Créer Une classe Compte caractérisée par son solde et un code qui est incrémenté lors de sa création ainsi que son propriétaire qui représente un client.

- Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe (le numéro de compte et le solde sont en lecture seule)
- Définir un constructeur permettant de créer un compte en indiquant son propriétaire.
- Ajouter à la classe Compte les méthodes suivantes :
  - Une méthode permettant de Crediter() le compte, prenant une somme en paramètre.
  - Une méthode permettant de Crediter() le compte, prenant une somme et un compte en paramètres, créditant le compte et débitant le compte passé en paramètres.
  - Une méthode permettant de Debiter() le compte, prenant une somme en paramètre
  - Une méthode permettant de Débiter() le compte, prenant une somme et un compte bancaire en paramètres, débitant le compte et créditant le compte

passé en paramètres
  - Une méthode qui permet d'afficher le résumé d'un compte.
  - Une méthode qui permet d'afficher le nombre des comptes crées.
- Créer un programme de test pour la classe Compte.

Exercice 6

Définir une classe Personne composée d'un nom, d'un prénom, et d'une adresse mail. Vous écrirez aussi les méthodes constructeurs, getter et setter des champs et redéfinissez la méthode toString().

Modifiez la classe Individu pour pouvoir :
- Afficher le nombre d'objets créés,
- Gérer la liste des Personnes créés,
- Afficher la liste des Personnes créés,

Exercice 7

‘ Créez la class Etudiant défini par : un nom, 2 semestres ayant chacun 8 notes, la moyenne de ses notes et un résultat qui vaut "non admis" si (moyenne < 12) sinon "admis". Définir la classe Etudiant en la dotant des méthodes nécessaires pour construire les objets et leur donnant la possibilité de calculer leurs moyennes et leurs résultats.

Exercice 8 : Attributs static et tableau

Créez deux class : une classe **Etudiant** et une deuxième classe qui s'appelle **Classe**.

La classe Etudiant a les attributs suivants :
- **CNE** : c'est un entier auto-incrémenté
- **nom** : String
- **prenom** : String
- **notes** : un tableau de 3 réels
- **autoIncrement** : entier (static) initialisé à 1 et incrémenté à chaque création d'un nouveau objet du type Etudiant.

La classe Etudiant a les méthodes suivantes :
- un constructeur avec paramètres : nom, prenom et 3 notes.
- Une méthode **calculerMoyenne()** qui calcule et retourne la moyenne d'un étudiant suivant la formule $0.4 * note1 + 0.4 * note2 + 0.2 * note3$.
- Une méthode **afficher()** qui affiche les informations de l'étudiant et sa moyenne.

Une classe est formée d'un ensemble d'étudiant dont le nombre ne doit pas dépasser un nombre d'étudiant maximum fixé à 30.

La classe **Classe** contiendra les attributs suivants :
- **nombre d'étudiant de la classe** : entier (initialisé à 0).
- **Nombre d'étudiant Max** : entier (static et constant).
- **Un tableau d'étudiants**

Les méthodes pour Classe sont :
- un constructeur sans paramètres.
- Une méthode **ajouterEtudiant(Etudiant e)** : qui ajoute l'étudiant donné en paramètre au premier élément libre du tableau d'étudiants, et incrémente le nombre d'étudiants de la classe.

— Une méthode **afficherClasse()** : qui fait appel à la méthode **afficher()** de la classe Etudiant pour afficher les étudiants d'une classe.

Ecrire un programme de test pour :

1. créer deux instances de la classe Etudiants
2.
3. créer une instance de Classe et lui affecter les deux étudiants.
4. Afficher la liste des étudiants de la classe suivant ce format :

CNE	**Nom**	**Prenom**	**Moyenne**
1	NOM 1	PRENOM 1	12.5
2	NOM 2	PRENOM 2	16.5

TP 3 : Class abstraite et héritage

Exercice 1

On cherche au cours de cet exercice à modéliser l'embarquement de véhicules dans un Carferry. Le paquetage `Ferry` contient les éléments suivants :

1. **Classe Personne (abstraite)** : Contient des informations sur les noms, les prénoms, la date de naissance et les adresses. Elle inclut une méthode abstraite `voyager()`.
2. **Classe Voyageur** : Hérite de la classe `Personne` et contient en plus un attribut désignant la date de voyage.
3. **Classe Conducteur** : Hérite de la classe `Voyageur` et contient en plus :
   — Un attribut désignant le numéro de permis.
   — Une méthode `ObtenirPermis()` qui, en fonction de l'âge de la personne (minimum 18 ans), permet de lever une exception en cas d'âge non réglementaire.
4. **Classe Véhicule** : Caractérisée par son modèle, son immatriculation, son kilométrage, son poids à vide, sa longueur (en mètres), et sa réserve de carburant (en litres).
   (a) Un véhicule peut être conduit par une personne adulte. Ajouter les instructions nécessaires pour qu'un véhicule puisse désigner son conducteur (objet de la classe `Conducteur`). *Question : Le même conducteur peut-il être le conducteur de plusieurs véhicules ?*
   (b) Écrire un constructeur précisant le modèle, l'immatriculation, la longueur et le poids. Les autres propriétés seront initialisées à 0. *Expliquez comment vous traitez le fait qu'à la construction, un véhicule n'a pas encore de conducteur.*
   (c) Écrire une méthode `ChangeConducteur()` pour changer le conducteur d'un véhicule.
   (d) Réécrire la méthode `toString()` indiquant les caractéristiques du véhicule et de son éventuel conducteur.
   (e) Écrire la méthode `GetPoids()` retournant le poids total (poids à vide + poids du carburant [1L = 1kg] + poids de l'éventuel conducteur).
   (f) Écrire la méthode `allerAlaPompe(float quantite)` pour ajouter du carburant et retourner la nouvelle quantité.
   (g) Écrire la méthode `rouler(float distance, float taux_consommation)`. Une exception sera levée si le véhicule n'a pas de conducteur ou si le carburant est insuffisant.
5. **Classe Camion** : Hérite de `Vehicule` et peut transporter une cargaison de colis.
   (a) Mémoriser la cargaison sous forme d'une collection d'objets de la classe `Colis`. Choisir un type de collection approprié en justifiant votre réponse.
   (b) Réécrire le constructeur pour initialiser la cargaison avec une collection vide.
   (c) Ajouter la méthode `int ajouter(Colis c)` qui retourne le nombre de colis après l'ajout.
   (d) Ajouter la méthode `boolean retirer(Colis c)` qui retourne vrai si le colis a été trouvé et retiré.
   (e) Réécrire la méthode `PoidCamion()` (somme des poids : véhicule vide, carburant, conducteur et cargaison).

(f) Réécrire la méthode `toString()` (caractéristiques du véhicule et de sa cargaison).

6. **Classe Voiture** : Hérite de `Vehicule` et transporte un nombre limité de passagers.
   (a) Définir le nombre maximum de passagers et mémoriser la liste des passagers (`Voyageur`).
   (b) Choisir un type de collection approprié et justifier.
   (c) Réécrire le constructeur pour définir la capacité maximale et initialiser la collection.
   (d) Ajouter la méthode `int monter(Voyageur p)` : lève une exception si la voiture est pleine, sinon ajoute le passager et retourne le nombre total.
   (e) Ajouter la méthode `boolean descendre(Voyageur p)` : retire un passager et retourne vrai si trouvé.
   (f) Réécrire la méthode `PoidVoiture()` incluant le poids des passagers.
   (g) Réécrire la méthode `toString()`.
7. **Classe CarFerry** : Hérite de `Vehicule`. Il transporte des voitures ou des camions répartis dans une cale (longueur et charge limitées) en deux piles (gauche et droite - LIFO).
   (a) Faire un schéma de la structure mémoire d'un objet `CarFerry`.
   (b) Écrire la classe `CarFerry`.
   (c) Réécrire le constructeur initialisant les limites de la cale et les piles de véhicules.
   (d) Réécrire la méthode `PoidsCarFerry()`.
   (e) Écrire la méthode `Embarquer(Vehicule v)` : empiler à droite ou à gauche pour équilibrer le poids, sans dépasser les limites de sécurité.
   (f) Écrire la méthode `Debarquer(Vehicule v)` : dépiler en maintenant l'équilibre.
   (g) Écrire une méthode `PropVoiture()` calculant la proportion de voitures à bord.
   (h) Écrire une méthode `Chercher(Voyageur p)` pour localiser une personne sur le ferry (conducteur ou passager).

TP 4 : Projet

Exercice 1 (Gestion des comptes bancaires)

Une agence bancaire a une adresse et un numéro d'agence incrémenté automatiquement. Un client ne peut avoir des comptes que dans une seule agence de la banque. Il a un code incrémenté automatiquement, un nom, un prénom et une adresse. A sa création, le client doit préciser son agence bancaire.

Un compte bancaire possède à tout moment une donnée : son solde. Ce dernier peut être positif (compte créditeur) ou négatif (compte débiteur). Chaque compte est caractérisé par un code et un solde à sa création : Son code est incrémenté automatiquement, tandis que le solde peut être nul ou initialisé par une somme. Doit être associé à son propriétaire (un client). Préciser l'agence bancaire auprès de laquelle le compte a été ouvert.

Utiliser son compte consiste à pouvoir y faire des dépôts et des retraits. Pour ces deux opérations, il faut aussi consulter le solde de son compte par la méthode getSolde().

Les comptes sont de nature différente selon qu'ils soient payants ou d'épargne :

- Un compte Epargne est un compte bancaire qui possède en plus un champ "TauxIntérêt=6" et une méthode calculIntérêt() qui permet de mettre à jour le solde en tenant compte des intérêts.
- Un compte payant est un compte bancaire pour lequel chaque opération de retrait et de versement est payante et vaut 5dh.

Travail à faire :

- Développer les différentes classes. Chaque classe doit contenir au moins un constructeur d'initialisation. A la création d'un objet donné, l'identifiant (code/numéro) doit être initialisé et incrémenté automatiquement selon le format suivant : $Identifiant = NomClasse : NumroObjet$. Exemples d'identifiants des deux premières instances de type Client : Client :1 et Client :2. Utiliser les méthodes getClasse() et getName() pour une initialisation automatique.
- Chaque classe doit redéfinir la méthode toString().
- Développer une classe ApplicationBancaire, dans cette classe on demande de :
  - Créer une seule agence bancaire.
  - Créer un tableau composé de 4 clients :
    - Un client ayant un compte d'épargne (solde = 1000dh).
    - Un client ayant un compte payant (solde = 2500dh).
    - Un client ayant 2 comptes payants (soldes 0dh et 3000dh).
    - Un client avec un compte d'épargne et un compte payant (soldes 2300dh et 0dh).
  - Permettre à un client de déposer une somme quelconque dans son/ses compte(s) en appelant la méthode deposer(...).
  - Permettre à un client de retirer une somme quelconque dans son/ses compte(s) en appelant la méthode retirer(...).
  - Définir la méthode addClient(...) permettant d'ajouter un client crée ainsi que ses comptes à l'agence. Utiliser cette méthode pour ajouter tous les clients crées.
  - Appliquer la méthode CalculIntérêt(...) sur tous les comptes d'épargne de l'agence.
  - Afficher les informations suivantes : Liste des différents clients avec leurs dif-

férents comptes, Liste des comptes d'épargne de l'agence, Liste des comptes payants de l'agence, Le solde total des comptes d'un client, Classement des différents clients selon le solde total de chacun.

Exercice 2 (Gestion des salaires pour une PME)

On souhaite développer un programme de calcul de salaire pour une entreprise PME, l'analyse du service « salaire » de cette entreprise a permis d'aboutir aux informations suivantes :

Information	**Description**
Matricule	Unique pour chaque employé et ne peut être modifié
Nom	Chaîne de caractères
Adresse	Chaîne de caractères
Salaire	Réel

— Donner la déclaration de la classe Employé Comportant au moins un constructeur et les méthodes d'accès.
— Cette entreprise comporte plusieurs types d'employés : Des employés qui sont payés suivant le nombre d'heures qu'ils ont travaillé dans la semaine. Ils sont payés à un certain tarif horaire. D'autres employés, payés au forfait par jour. Les commerciaux sont payés avec une somme fixe pour la semaine.
— Donnez les classes qui héritent de la classe Employe et redéfiniront la méthode setSalaire() pour chaque type des employés. Les classes comporteront deux constructeurs : un qui ne prend en paramètre que le matricule, le nom et l'adresse de l'employé et l'autre qui prend en paramètres en plus toutes les informations pour le calcul du salaire.
— Une classe Paie comportera une unique méthode main() qui entrera les informations sur des employés des différents types (3 commerciaux, 2 de type i.et 2 de type ii.). Les employées seront enregistrées dans un tableau Employé.
— Au moins un des employés sera créé avec le constructeur ne possédant comme paramètres que le Matricule, le Nom et l'Adresse et vous entrerez ensuite les informations pour son salaire avec la méthode setSalaire(). Pour un autre employé, vous utilisez le constructeur pour entrer les informations sur le salaire.
— Écrire une méthode qui affichera le salaire hebdomadaire de chacun des employés et la masse salariale Hebdomadaire des employés enregistrés dans le tableau.

TP 4 : Accès Base de Données (JDBC)

Soit le modèle relationnel suivant relatif à la gestion des notes annuelles d'une promotion d'étudiants :

— ETUDIANT(CodeEtudiant , NomEtudiant, ddnEtudiant)
— MATIERE(CodeMat , NomMatiere, CoefMatiere)
— ENSEIGNANT(CodeEns , NomEns, GradeEns, #CodeMat)
— NOTE (#CodeEt, #CodeMat , note)

Travail à faire :

1. Créez la base de données et y insérer quelques lignes de test
2. Établissez une connexion JDBC à une base de données MySQL.
3. Exécutez une requête SELECT pour récupérer tous les étudiants de la table ETUDIANT.
4. Insérez un nouvel étudiant dans la table ETUDIANT.
5. Mettez à jour le coefficient d'une matière dans la table MATIERE.
6. Supprimez un enseignant de la table ENSEIGNANT.
7. Exécutez une requête JOIN pour récupérer les informations des étudiants avec leurs notes.
8. Recherchez et affichez les matières enseignées par un enseignant spécifique.
9. Calculez et affichez la moyenne des notes pour un étudiant spécifique.
10. Insérez une nouvelle note pour un étudiant dans la table NOTE.
11. Mettez à jour le nom d'un étudiant dans la table ETUDIANT.
12. Supprimez toutes les notes pour un étudiant spécifique dans la table NOTE.
13. Listez tous les étudiants inscrits dans une matière spécifique.
14. Affichez tous les enseignants qui enseignent une matière spécifique.
15. Calculez et affichez le nombre d'étudiants inscrits dans chaque matière.
16. Listez les matières pour lesquelles un étudiant a obtenu une certaine note minimum.
17. Récupérez et affichez les informations des enseignants et les matières qu'ils enseignent.
18. Insérez une nouvelle matière dans la table MATIERE.
19. Mettez à jour le grade d'un enseignant dans la table ENSEIGNANT.
20. Supprimez une matière de la table MATIERE.
21. Affichez les étudiants qui ont une date de naissance après une certaine date.

Troisième partie
Modélisation UML

TD 1 : Diagramme de cas d'utilisation

Exercice 1 :

Dans un établissement scolaire, on désire gérer la réservation des salles de cours ainsi que du matériel pédagogique (ordinateur portable ou/et vidéo projecteur). Seuls les enseignants sont habilités à effectuer des réservations (sous réserve de disponibilité de la salle ou du matériel). Le planning des salles peut quant à lui être consulté par tout le monde (enseignants et étudiants). Par contre, le récapitulatif horaire par enseignant (calculé à partir du planning des salles) ne peut être consulté que par les enseignants. Enfin, il existe pour chaque formation un enseignant responsable qui seul peut éditer le récapitulatif horaire pour l'ensemble de la formation.

Question : Donner le diagramme de cas d'utilisation correspondant.

Exercice 2 :

Les inscriptions dans une école fonctionnent de la façon suivante : Au début de chaque semestre, un catalogue des cours proposés est fourni par la scolarité aux étudiants. Chaque cours est décrit par un certain nombre d'informations, en particulier, l'enseignant, le cursus et les pré-requis. Ce catalogue ne peut être créé avant que tous les cours ne soient affectés à des enseignants. Pour cela, chaque enseignant accède au système d'inscription pour indiquer les cours qu'il prévoit d'enseigner. Les étudiants doivent remplir des fiches d'enregistrement qui indiquent leurs choix de cours. L'étudiant standard doit suivre 4 enseignements choisis dans le catalogue. Il devra indiquer aussi deux cours supplémentaires. En effet, il se peut que, parmi les 4 cours choisis, l'un des cours soit trop plein ou abandonné par manque d'étudiants. Chaque cours doit en effet être dispensé à au moins 5 étudiants et au plus 30 étudiants. Si un cours est choisi par moins de 5 étudiants, il est supprimé. Ces fiches sont gérées par la scolarité. Une fois la période d'inscription terminée, un programme est exécuté pour affecter les étudiants aux cours. Dans la plupart des cas, les étudiants obtiennent ce qu'ils ont choisi. Après que tous les étudiants aient été correctement affectés aux différents cours, un listing est imprimé pour chaque étudiant pour vérification. Une fois la sélection de cours d'un étudiant validée, l'information est transmise au système de facturation qui facturera l'étudiant pour son semestre.

Questions : On souhaite analyser les besoins de ce système d'inscription.

1. Donner les acteurs de ce système.
2. Donner le diagramme des cas d'utilisation du système.

Exercice 3 :

On s'intéresse à une société qui fabrique des composants de moteurs. Sur le plan des moyens de production, la société dispose de 6 usines dont une dédiée au montage. La fabrication comporte une phase d'usinage, une phase de traitement thermique et de surface, puis de montage des sous-ensembles qui s'intégreront dans les moteurs. La direction commerciale reçoit chaque jour les commandes clients. Compte tenu des délais de fabrication (cycles de 8 à 15 mois), les clients pressés peuvent passer des commandes prioritaires

mais au prix d'une surfacturation de 20% de la commande. Les usines les traitent alors en priorité. Les gros clients (plus de 10 commandes par an) sont traités en priorité aussi. Après un contrôle technique des articles commandés, la direction commerciale introduit la commande en ordinateur et obtient en sortie, l'éclatement des ensembles composés en composants élémentaires qui appartiennent à deux catégories distinctes :

— Pièces fabriquées par l'entreprise ;
— Pièces sous-traitées.

Elle imprime alors un listing de composants "maison" à destination des usines et un listing des composants soustraités à destination du service de gestion du stock de l'entreprise pour approvisionnement. Ce service gère aussi l'approvisionnement bimensuel en matières premières. La commande globale est transmise à l'usine de montage qui ne peut finaliser la commande qu'après réception de l'ensemble des composants requis. Des modifications (avenants) aux commandes clients peuvent être apportées après l'enregistrement de la commande, afin d'amender :

— Les codes articles (dans le cas d'une évolution technique de l'article), ou les autres caractéristiques de l'article (délai, quantité commandée...).

La direction commerciale se charge aussi du suivi de la facturation. Les coûts réels sont comptabilisés par commande reçue, avec une gestion du nombre d'heures passées (heures productives transmises par les usines) et de la valeur (prix d'achat) des matières premières ou pièces semi-finies en provenance de fournisseurs divers (fournie par le service de gestion du stock). La facture est alors transmise à l'usine de montage qui gère aussi la livraison de la commande.

Question : Donner le diagramme des cas d'utilisation de ce système.

TD 2 : Diagramme de classes

Exercice 1 :

Des interviews d'experts métier ont permis de mettre en évidence les éléments suivants :

- Un vol a un aéroport de départ et un aéroport d'arrivée ;
- Un vol a une heure de départ et une heure d'arrivée, ainsi qu'une date de départ et une d'arrivée ;
- Un vol peut comporter des escales dans des aéroports ;
- Les escales interviennent dans un ordre déterminé ;
- Une escale a une heure d'arrivée et une heure de départ ; Chaque aéroport a un nom ;
- On peut ouvrir (et fermer) à la réservation chacun des vols.

Question : Proposez un diagramme de classes pour modéliser les éléments ci-dessus en utilisant une classe association. N'oubliez pas les attributs et les cardinalités des associations.

Exercice 2 :

La société C fabrique des pièces métalliques réalisées dans son atelier. La société C est en relation commerciale uniquement avec des clients réguliers. Les factures sont mensualisées : toutes les commandes d'un client dans le mois sont facturées à la fin du mois. A la réception d'une commande, le secrétariat de la société C édite un bon de fabrication qu'elle transmet à l'atelier (un double de ce bon de fabrication est envoyé au client). Une fois la pièce fabriquée, l'atelier complète le bon de fabrication et le transmet au secrétariat qui enregistre la date de fin de fabrication et informe le client que sa pièce est disponible à l'atelier. A la fin du mois, les factures sont éditées. Une facture est éditée pour chaque client ayant effectué une commande dans le mois. Si une facture n'est pas payée dans le délai de 10 jours ouvrables alors un courriel de relance est envoyé au client. Tant qu'il n'aura pas payé sa facture aucune de ses commandes ne sera acceptée chaque fois qu'il effectuera une commande, un courriel de refus lui sera envoyé. Une facture payée est archivée 15 ans et ensuite détruite.

Hypothèse : nous considérons que tous les paiements sont corrects : un client paye le total facturé ou il n'envoie pas de règlement. *Hors Contexte :* l'enregistrement de nouveau client est hors contexte.

Question : Donner le diagramme de classes de cette application.

Exercice 3 :

Une académie souhaite gérer les cours dispensés dans plusieurs collèges. Pour cela, on dispose des renseignements suivants :

- Chaque collège possède d'un site Internet ;
- Chaque collège est structuré en départements, qui regroupent chacun des enseignants spécifiques.
- Parmi ces enseignants, l'un d'eux est responsable du département ;
- Un enseignant se définit par son nom, prénom, tél, mail, date de prise de fonction et son indice ;
- Chaque enseignant ne dispense qu'une seule matière ;

— Les étudiants suivent quant à eux plusieurs matières et reçoivent une note pour chacune d'elle ;
— Pour chaque étudiant, on veut gérer son nom, prénom, tél, mail, ainsi que son année d'entrée au collège ;
— Une matière peut être enseignée par plusieurs enseignants mais a toujours lieu dans la même salle de cours (chacune ayant un nombre de places déterminé) ;
— On désire pouvoir calculer la moyenne par matière ainsi que par département ;
— On veut également calculer la moyenne générale d'un élève et pouvoir afficher les matières dans lesquelles il n'a pas été noté ;
— Enfin, on doit pouvoir imprimer la fiche signalétique (nom, prénom, tél, mail) d'un enseignant ou d'un élève.

Question : Elaborez le diagramme de classes correspondant. Pour simplifier l'exercice, on limitera le diagramme à une seule année d'étude.

TD 3 : Diagramme d'objets

Exercice 1 :

Considérez le diagramme de classes de gestion d'un hôtel :

[Insérer l'image du diagramme de classes de l'hôtel ici]

Questions : En vous basant sur le précédant diagramme de classe faite un diagramme d'objet qui représente cette situation : L'hotel matignon dont le gérant Mr Formul Alain s'occupe, possède 50 chambres. L'une des chambres est louée à Mr Lelore.

Exercice 2 :

Une équipe d'informatique est composée de développeurs. Elle travaille pour une entreprise et possède un identifiant et un logo. Un développeur est caractérisé par le numéro de sa carte d'identité, son nom, son prénom et son émail, il utilise un ordinateur qui lui est personnel. Un développeur peut être un programmeur spécialisé dans un ou plusieurs langages de programmation, ou un concepteur expert dans une méthode ou langage de modélisation. Les développeurs écrivent le code. Les concepteurs dialoguent avec les utilisateurs et créent les modèles.

Questions :

1. Présentez le diagramme de classes relatif à la description ci-dessus.
2. Créer un diagramme d'objets correspondant au texte suivant : Ali et Zied sont des programmeurs spécialisés respectivement dans les langages Ada et Java. Anis est un concepteur UML. Ils font partie de l'équipe DEV qui représente l'entreprise Tech-Soft. Le logo de l'équipe DEV est DEV +. Tous les développeurs utilisent un seul ordinateur.

TD 4 : Diagramme de séquences

Exercice 1 :

Le déroulement normal d'utilisation d'une caisse de supermarché est le suivant :

- Un client arrive à la caisse avec ses articles à payer
- Le caissier enregistre le numéro d'identification de chaque article, ainsi que la quantité si elle est supérieure à 1
- La caisse affiche le prix de chaque article et son libellé
- Lorsque tous les achats sont enregistrés, le caissier signale la fin de la vente
- La caisse affiche le total des achats
- Le caissier annonce au client le montant total à payer
- Le client choisit son mode de paiement
  - Liquide : le caissier encaisse l'argent, la caisse indique le montant à rendre au client
  - Chèque : le caissier note le numéro de pièce d'identité du client
  - Carte de crédit : la demande d'autorisation est envoyée avant la saisie
- La caisse enregistre la vente et l'imprime
- Le caissier donne le ticket de caisse au client

Questions : Modéliser cette situation à l'aide d'un diagramme de séquence en ne prenant en compte que le cas du paiement en liquide.

Exercice 2 :

Un organisme de sécurité sociale souhaite développer une application gérant des patients, leur médecin traitant et des médecins spécialistes. Une personne est soit un assuré, soit un médecin, un médecin pouvant être malade et être assuré. Un médecin est soit généraliste, soit spécialiste, un généraliste ne pouvant pas être spécialiste et inversement. Un patient a choisi ou non son médecin traitant. Un patient consulte, à une date donnée, un généraliste. Lors des consultations, le généraliste peut ou non prescrire une ou plusieurs consultations chez un spécialiste. Cette application peut être utilisée par l'organisme de sécurité sociale pour inscrire un assuré, enregistrer un médecin traitant pour un assuré, enregistrer des feuilles de maladie et effectuer des remboursements. Tout médecin peut également, via l'application, enregistrer une feuille de maladie, prescrire des médicaments et prescrire une consultation chez un spécialiste.

Questions : Donnez les diagrammes de séquences des deux cas d'utilisations suivantes :

1. Enregistrer un médecin traitant
2. Effectuer un remboursement- cas d'une réception de feuille de soins papier.

Exercice 3 :

Un vidéo club est un centre de distribution qui assure essentiellement la location de films préenregistrés. Les éditeurs procurent les cassettes aux exploitants soit en location soit en vente. Les exploitants peuvent donc passer avec les éditeurs des contrats de location d'une durée moyenne de 6 mois ou passer des commandes à partir de catalogues fournis régulièrement par les éditeurs. Un vidéo club entretient des relations avec une trentaine d'éditeurs environ. Lorsque les exploitants constatent une usure des cassettes qui leur appartiennent, ils ont la possibilité de les vendre à des grossistes qui peuvent alors

pratiquer des ventes au rabais. Un seul statut est proposé aux clients, celui d'adhérent. Chaque adhérent se voit attribuer une carte d'adhésion sur laquelle est mentionné un code adhérent. Il peut alors choisir entre plusieurs types d'abonnement. Les tarifs varient selon le mode d'abonnement choisi. Quatre tarifs adaptés aux locations sont proposés en fonction des différents types d'abonnement. Toutefois, on peut louer des cassettes aux clients non abonnés sans leur faire profiter des avantages tarifaires réservés aux abonnés.

Questions :

1. Donner le diagramme des cas d'utilisation du système.
2. Décrire le scénario Location Cassettes par un diagramme de séquence.

TD 5 : Diagrammes d'états de transitions/activités

Exercice 1 :

Considérons une classe Partie dont la responsabilité est de gérer le déroulement d'une partie de jeu d'échecs. Cette classe peut être dans deux états :
- Le tour des blancs ;
- Le tour des noirs.

Les événements à prendre en considération sont :
- Un déplacement de pièces de la part du joueur noir ;
- Un déplacement de pièces de la part du joueur blanc ;
- La demande de prise en compte d'un échec et mat par un joueur ;
- Un échec et mat assure la victoire du dernier joueur. Dans ce cas, une activité noirsGagnants ou blancsGagnants est déclenchée ;
- La demande de prise en compte d'un pat qui mène à une fin de partie, avec une égalité. Dans ce cas, une activité égalité est déclenchée.

Questions : Représenter le diagramme d'états/transitions associé à la classe Partie.

Exercice 2 :

On considère le fonctionnement d'un distributeur de billets. Le client introduit sa carte dont la validité est immédiatement vérifiée. Il est ensuite invité à saisir le code de la carte. Après trois tentatives infructueuses, la carte est avalée. Sinon le client peut indiquer le montant qu'il désire retirer, le solde de son compte bancaire est alors consulté pour s'assurer que le retrait est possible. En cas de solde insuffisant, le client en est informé et peut alors saisir un montant inférieur. Si le solde du compte est suffisant, le distributeur restitue la carte et délivre alors les billets accompagnés d'un reçu.

Question : Modélisez le fonctionnement du distributeur à l'aide d'un diagramme d'activités.

Exercice 3 :

La description de l'utilisation du logiciel de saisie d'ordres de mission et de remboursement des frais de mission est la suivante :

1. Avant un déplacement, un salarié de l'entreprise saisit un ordre de mission. Il saisit les données décrivant la mission. Il peut saisir un ou plusieurs trajets.
2. Le responsable hiérarchique valide (ou pas) l'ordre de mission.
3. Après avoir effectué la mission, le salarié saisit une demande le remboursement des frais de mission. Il indique le montant des frais engagés pour la mission et fournit les justificatifs associés (documents scannés).
4. Le service comptabilité rembourse les frais de la mission au salarié.

Le logiciel offre deux possibilités de paramétrage :
- Chaque salarié saisit les données concernant sa voiture, s'il en a une marque, modèle, immatriculation et puissance fiscale.
- Le service des ressources humaines indique le responsable hiérarchique pour chaque salarié (s'il en a un).

Le logiciel ne nécessite pas de login. Celui-ci est pris en charge par l'intranet de l'entreprise.

Questions :

1. Construire un diagramme d'états représentant les états d'un ordre de mission et de la demande de remboursement associée en faisant apparaître :
   — cinq états à valider, validé, non validé, à rembourser, remboursé,
   — une décision,
   — les transitions dont le libellé indique un événement et/ou une condition.
2. Construire un diagramme d'activité présentant le processus métier global et contenant :
   — trois partitions : un salarié, son responsable hiérarchique et le service de comptabilité,
   — un point de départ et un point d'arrivée,
   — quatre étapes correspondant aux cas d'utilisation du logiciel (hors paramétrage), une étape effectuer la mission,
   — cinq objets : Ordre de mission (chacun avec un des états indiqués dans la question précédente),
   — les flots de contrôles et les flots d'objet entre les éléments précédents.

TD 6 : Diagramme de collaboration

Exercice 1 :

Vous devez modéliser un système de réservation de billets de train. Nous décrivons ci-dessous l'interaction entre ce système et un voyageur qui désire acheter un billet à partir d'un automate mis à disposition des voyageurs dans la gare. Le voyageur commence par choisir l'option Demande de réservation sur la borne interactive du système. Celle-ci lui offre alors la possibilité d'entrer les paramètres de sa réservation. Le voyageur ne doit pas tout remplir, mais certains de ces critères sont obligatoires. Le voyageur confirme sa requête en appuyant sur le bouton correspondant. La borne interactive envoie alors la requête au serveur central. Celui-ci analyse la demande de réservation et envoie une réponse à la borne interactive.

— Si la réponse est une Requête refusée, la borne propose à l'utilisateur de recommencer en appuyant sur le bouton Demande de réservation.
— Si la réponse est une Requête partiellement satisfaite, la borne affiche les critères qui n'ont pus être honorés, ainsi que le billet proposé, et propose à l'utilisateur d'acheter le billet correspondant ou de procéder à une nouvelle demande de réservation.
— Si la réponse est une Requête satisfaite, le système invite le client à procéder à l'achat du billet correspondant.

À tout moment, le client peut annuler l'opération en cours. Si l'utilisateur choisit d'acheter le billet, la borne lui demande d'insérer sa carte bancaire et de taper son code. La borne prend contact avec un service d'autorisation de paiement. Si celui-ci autorise la transaction, la borne envoie une confirmation au serveur et imprime le billet.

Question : Donner le diagramme de collaborations décrivant les détails de conception du cas d'utilisation Réservez un billet de train.

Exercice 2 :

On souhaite gérer les différents objets qui concourent à l'activité d'un magasin de vente de fleurs.

— Le client demande au vendeur des renseignements des renseignements sur les compositions florales ;
— Le vendeur lui fournit toutes les informations nécessaires ;
— Le client commande alors la composition de son choix et le vendeur émet le bon de fabrication qu'il transmet à son ouvrier fleuriste ;
— Le vendeur édite ensuite la facture correspondante ; L'ouvrier fleuriste crée la composition puis archive le bon de fabrication : Il remet alors la composition au vendeur ;
— La facture est remise au client pour règlement une fois le bouquet réalisé ;
— Une fois la facture réglée, le client récupère sa composition et quitte le magasin.

Question : Modéliser cette situation à l'aide d'un diagramme de collaboration.

Exercice 3 :

Un club de sport souhaite utiliser un logiciel pour faire la gestion de ses différentes activités : inscription des membres, inscriptions aux cours, facturation, calcul de statistiques, établissement de bilan financier, etc. Un analyste a déjà fait l'étude des cas d'utilisation

de ce système. Il a également préparé un diagramme de classes modélisant les données du système. Vous êtes le concepteur et devez faire le design du cas d'utilisation Inscription d'un membre à un cours privé voici la description détaillée de ce cas d'utilisation.

1. Sur la page d'inscription, le préposé entre le type de cours (tennis, musculation, etc.) auquel le membre souhaite s'inscrire ainsi que le nom du professeur.
2. Le système cherche le professeur dans le répertoire des profs puis cherche une plage de cours libre, c'est-à-dire une tranche horaire où le professeur est disponible pour offrir un cours (on suppose que le système a un moyen pour identifier les disponibilités du prof.) et qui ne chevauche pas celle d'autres plages de cours déjà réservées avec ce professeur.
3. Si une tranche horaire est disponible, le système ajoute une nouvelle plage de cours au planning du professeur pour cet horaire. (La plage de cours est réservée en attendant la confirmation du membre). Le système affiche le résultat de la requête (i.e. cours, professeur, horaire proposé) sur la page d'inscription, dans une zone prévue à cette effet.
4. Si la plage de cours proposée convient au membre, le préposé complète la page d'inscription en entrant le nom du membre dans l'espace prévu. Il clique ensuite sur le bouton Confirmer.
5. Pour procéder à l'inscription, le système cherche d'abord le membre dans le répertoire des membres. On assigne alors à ce membre la plage de cours en question.
6. Le cours est facturé au membre. Inclure Facturation d'un cours.
7. Le système confirme l'inscription en affichant un message au bas de la page d'inscription.

Question : Donnez les diagrammes de séquences et de collaboration.

TD 7 : Diagrammes de composants et déploiement

Exercice 1 :

Un producteur produit des objets qui sont consommés par un consommateur. A une lointaine époque, le consommateur pouvait demander directement l'objet au producteur. Toutefois, la législation actuelle, pour des raisons d'hygiène, impose que le producteur ne soit plus en contact direct avec le consommateur. L'échange d'objets se fait donc par un compartiment muni de deux trappes. Une trappe est destiné au producteur, l'autre au consommateur. Il ne peut y avoir qu'une seule trappe ouverte en même temps. Le producteur, son objet terminé au bout d'un temps plus ou moins long, ouvre la trappe et met l'objet dans le compartiment. Le consommateur, quant à lui, ouvre la trappe dès qu'il peut pour voir si l'objet de sa convoitise se trouve dans le compartiment.

Nous nous proposons de simuler par un programme Java le comportement d'un producteur et d'un consommateur qui s'échangent des objets via un compartiment. Afin de simplifier la simulation, nous décidons que le nombre d'objets échangés sera de 10. Le producteur ainsi que le consommateur peuvent être assimilés à des threads Java au comportement bien déterminé.

Questions : Proposer un diagramme de composants et un diagramme de déploiement pour la description ci-dessus.

Exercice 2 :

On considère une application constituée des fichiers suivants :
- Un code source `registre.cpp`
- Un programme exécutable `registre.exe`
- Des librairies dynamiques `personne.dll` et `cours.dll`. Les librairies à liens dynamiques sont utilisées lors de l'exécution d'une application.

Question : Donnez le diagramme de composants correspondant.

Exercice 3 :

Une architecture matérielle est constituée d'un serveur, d'un kiosques et d'une console. Le serveur utilise une tour de disques durs montés en RAID. La console communique avec le serveur par une liaison Ethernet 100b et la console par une liaison RS232C. Les caractéristiques du serveur sont :
- Processeur Pentium ;
- Mémoire vive : 4 Go ;
- Ecran 17 pouces.

On peut aussi préciser les modules exécutables sur chaque noeud :
- Sur le kiosque : `user.exe` ;
- Sur le serveur `dbadmin.exe` et `tkmstr.exe` ;
- Sur la console : `admin.exe` et `config.exe`.

Question : Représentez le diagramme de déploiement correspondant.